

1. The assignment is to implement in code the extended Euclidean algorithm in finite fields. To do this, you will need to implement the addition, subtraction, multiplication, and division/modulus operations in finite fields. Include your code and a short write-up, demonstrating each of the operations. Try inverting the polynomial from the last homework: $x^7 + x^5 + x^4 + x^1$.

In order to come up with a code that performs the extended Euclidean algorithm for the Galois Field we did the following steps:

First, we used a higher-level programming language to implement the polynomial extended GCD algorithm over the integers mod p . For this program, we used built in function such as integer addition, subtraction, multiplication, division, the greatest integer function, and the mod p function for p a prime, as well as other procedural commands.

Second, we use our program to find the remainder (x) from the polynomial GCD of $x^8 + x^4 + x^3 + x + 1$ and $x^7 + x^5 + x^4 + x^1$ over the integers mod 2. As well as the polynomials $a(x)$ and $b(x)$ mod 2 such that $(x^8 + x^4 + x^3 + x + 1)b(x) + (x^7 + x^5 + x^4 + x^1)a(x) = \text{remainder}(x) \text{ mod } 2$

Third, using the above procedure we thought of how to generate a Galois Field given the integers and polynomials. The steps to find the Galois Field are:

- Find the upper bound of the finite field we want to generate. In the case of generating finite fields with size of a prime number p , we just need to use p as the upper bound. Since integer mod operation is easy to do, it's fairly simple to have a finite field of size p .
- In the case of generating finite field of size p^n , things get more complicated only because the basic arithmetic operations, such as addition, subtraction, multiplication, division, and mod on polynomial are more difficult to do. If we can find ways to do these operations, we are effectively changing the polynomials to an integral domain just like the integers.
- With the above approach we have created the following classes: GaloisField.java, GCD.java, Integral.java, InverseGF.java, Operations.java, Poly.java. Operations.java is an abstract representation for all integral operations. It is intended to unify the abstract class for the Integral and Poly class, which are the actual classes that implements the required arithmetic operations. Then, the class InverseGF.java will wrap around the two integral domains and create a Galois Field upon them.

Finally, the GCD class does the extended GCD algorithm on the two integral domains using the abstract class for all basic arithmetic operations.

Output:

Inverse Problem Calculation:

Extended Euclidean Algorithm Approach:

$\gcd(x, y) = 1$

$\gcd$, where $x*a + y*b = 1$

where $a = x^{-1}$ and $b = y^{-1}$

Input Polynomial Values:

$x = x^8 + x^7 + x^4 + x^3 + x + 1$

$y = x^7 + x^5 + x^4 + x$

Devision Remainder = $x + 1$

$b = x^5 + x^3 + x^2 + 1$

$a = x^4 + x^3 + x^2 + x + 1$

Inverse Input Binary Value:

0 0 0 1 1 1 1 1

Matrix:

10001000

11000000

11100000

11110000

11111000

01111000

00111000

00011000

Matrix * Inverse

Output:

0

1

0

1

1

1

0

0

1

1

0

1

1

0

0

0

Source Code: For a better look check InverseGF.java file

```
public class InverseGF {
    static int[][] matrix = {
        {1, 0, 0, 0, 1, 0, 0, 0},
        {1, 1, 0, 0, 0, 0, 0, 0},
        {1, 1, 1, 0, 0, 0, 0, 0},
        {1, 1, 1, 1, 0, 0, 0, 0},
        {1, 1, 1, 1, 1, 0, 0, 0},
        {0, 1, 1, 1, 1, 0, 0, 0},
        {0, 0, 1, 1, 1, 0, 0, 0},
        {0, 0, 0, 1, 1, 0, 0, 0}};

    public static void main ( String [ ] args ) {
        System.out.println("\n Inverse Problem Calculation:");
        System.out.println(" Extended Euclidean Algorithm Approach:");
        System.out.println(" gcd(x,y) = 1");
        System.out.println(" gcd, where x*a + y*b = 1\n where a = x^-1 and b = y^-1\n");

        Integral N = new Integral (2) ;
        System.out.println("\n Input Polynomial Values:");
        GaloisField<Integral> zero = new GaloisField<Integral>(N) ;

        Poly p1 = new Poly ( zero ) ;
        p1.setCoefficients( 1, 1, 0, 1, 1, 0, 0, 1, 1) ;
        System.out.println ( " x = " + p1 ) ;
        Poly p2 = new Poly ( zero ) ;
        p2.setCoefficients (0, 1, 0, 0, 1, 1, 0, 1, 0) ;
        System.out.println ( " y = " + p2 ) ;

        // Carry out the extended GCD algorithm .
        GCD<Poly> GCD = new GCD<Poly>() ;
        List<Poly> results = GCD.gcd(p1, p2) ;
        Poly d = results.get (0) ;
        Poly g = results.get (1) ;
        Poly h = results.get (2) ;

        System.out.println( "\n Division Remainder = " + d) ;
        System.out.println( " b = " + h) ;
        System.out.println( " a = " + g) ;
        int[] inverse = {0, 0, 0, 0, 0, 0, 0, 0};

        int[][] temp = new int[matrix.length][matrix.length];
        int[] key = {1, 1, 0, 0, 0, 1, 1, 0};

        int x = 0;
        for( GaloisField<Integral> i: g.getCoefficients()){
            inverse[x++] = Integer.parseInt(i.toString());
        }
        System.out.println("\n Inverse Input Binary Value: ");

        for(Integer i: inverse ){
            System.out.print(" " +i);
        }

        System.out.println("\n\n Matrix:");
        for( int x1 =0; x1 < 8; x1++){

            System.out.print(" ");
            for( int y=0; y < 8; y++){

                temp[x1][y] = matrix[x1][y]*inverse[y];
                System.out.print(temp[x1][y]);

            }
            System.out.println();
        }

        int tp = 0;
        int output1[] = new int[8];
        System.out.println("\n Matrix * Inverse");
        for( int x1 =0; x1 < matrix.length; x1++){

            tp = xor(temp[x1][0], temp[x1][1]);
            for( int y=2; y < matrix.length; y++){

                tp = xor(tp, temp[x1][y]);

            }
            output1[x1] = tp;
            System.out.print(" " + tp);
            System.out.println();
        }

        tp = 0;
        int output2[] = new int[8];
        System.out.println("\n Output:");
        for( int x1 =0; x1 < matrix.length; x1++){
            tp = xor(output1[x1], key[x1]);
            System.out.print(" " + tp);
            output2[x1] = tp;
            System.out.println();
        }

        public static int xor(int x, int y){

            if( x == y)
                return 0;
            else
                return 1;
        }
    }
}
```